

Heart Disease MLOps - Brief Report

Heart Disease Classification - End-to-End MLOps Pipeline

MLOps Assignment-I | Final Report (10-page brief) | CSCDZG548

Dataset	UCI Heart Disease (4 sources, 920 records)
Task	Binary classification - presence of heart disease (<code>num > 0</code>)
Stack	Python 3.11, scikit-learn, MLflow, Flask, GitHub Actions, Docker, Kubernetes (<code>kind</code> + <code>ingress-nginx</code>), Prometheus, Grafana
Best model	Random Forest, ROC-AUC = 0.914 on the 184-row hold-out test set
Code repository	https://github.com/2025CS05012bitspilani/heart_disease_classification_mlops.git

1. Abstract & Setup

End-to-end MLOps pipeline around the UCI Heart Disease dataset. Four heterogeneous source files are cleaned and harmonised, three classifiers are tuned with 5-fold CV, every experiment is tracked in MLflow, the winning pipeline is packaged as a `joblib` artefact + an MLflow model directory, served by Flask + gunicorn inside a Docker image, deployed onto a local Kubernetes cluster (`kind`) behind an `ingress-nginx` Ingress with an HPA, and instrumented with Prometheus metrics and structured JSON access logs. A four-stage GitHub Actions workflow lints, tests (35 cases, 74 % coverage), retrains, and smoke-tests the container on every push.

A new machine reaches a working `/predict` endpoint in three commands:

```
git clone https://github.com/2025CS05012bitspilani/heart_disease_classification_mlops.git
cd heart_disease_classification_mlops
python3 run_pipeline.py           # Tasks 1-9 end-to-end
```

`run_pipeline.py` auto-detects PEP 668 / unsupported Python versions and bootstraps a local `.venv` from the highest available `python3.10-3.12`. Optional tooling (`docker` , `kind` , `kubectrl` , `pandoc` , `npx`) is required only for Tasks 6-9.

Manual step-by-step fallback (when `run_pipeline.py` cannot run). Each command maps 1:1 to a named step of the orchestrator (`run_pipeline.py --only <step>`):

```

python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt # bootstrap
python -m src.data.download # Task 1 (--only data)
python -m src.data.preprocess # Task 2 (--only data)
python -m src.models.train # Task 3+4 (--only train)
pytest -q # tests (--only tests)
docker build -f docker/Dockerfile -t heart-api:latest . # Task 6 (--only docker)
bash scripts/demo_up.sh # Task 7 (--only k8s)
mlflow ui --backend-store-uri "file://$PWD/mlruns" --port 5500 & # Task 3 UI (--only mlflow_ui)
( cd monitoring && docker compose up -d ) # Task 8 (--only monitor)
bash reports/build_report.sh # Task 9 (--only report)

```

2. Problem Statement & Dataset

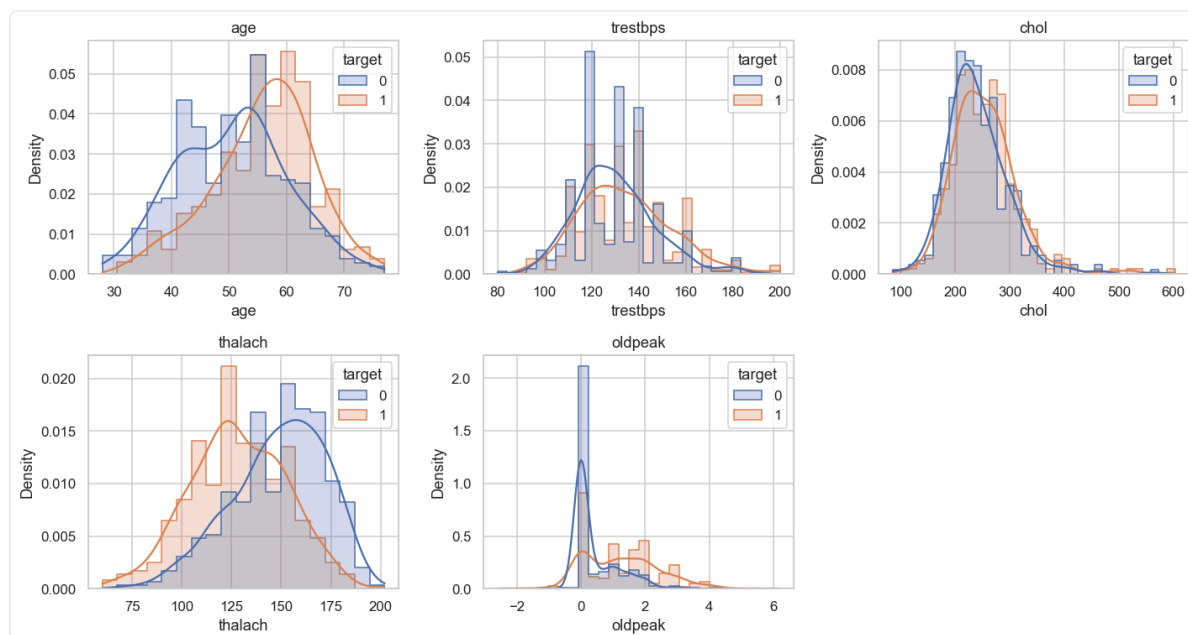
Coronary heart disease is a leading cause of mortality. Inexpensive, routinely-collected clinical features (age, blood pressure, cholesterol, ECG outputs) can triage patients for confirmatory testing. We frame the task as binary classification: given 13 attributes, predict whether the patient has any degree of heart-disease narrowing (`num > 0`). The deliverable is **the full lifecycle**, not the model alone.

Source	Records
Cleveland	303
Hungarian	294
Long-Beach VA	200
Switzerland	123
Total	920

Features (13): `age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal` . The raw `num` field (0-4) is binarised to `target = 1{num > 0}` . Target balance is 509 / 411 (55.3 % / 44.7 %), so we optimise ROC-AUC. Cleaning (`src/data/preprocess.py`): replace UCI's `?` with `NaN` ; treat sentinel zeros in `chol` / `trestbps` as missing; preserve numeric `NaN` s; add a `ca_missing` indicator (~66 % of non-Cleveland rows lack `ca`); persist to `data/processed/heart_disease_clean.csv` .

3. Exploratory Data Analysis

EDA artefacts live under `reports/figures/` . The target is mildly imbalanced, so no resampling is needed; ROC-AUC is the optimisation target.



Class balance and feature histograms split by target

`oldpeak - slope` , `cp - exang` and `thalach - age` are the strongest pairwise relationships (`correlation_heatmap.png`) - this guides the use of one-hot encoding for categoricals and tree-based models that handle correlated inputs well. The four UCI source files have very different missingness profiles (`missingness_per_source.png`), which directly drives the per-column `SimpleImputer` choices in §4 and the `ca_missing` indicator.

Figure	Modelling decision it informs
<code>class_balance.png</code>	Stratified 80/20 split, no SMOTE; ROC-AUC objective
<code>histograms_by_target.png</code>	<code>thalach</code> and <code>oldpeak</code> retained as-is (clear separation)
<code>correlation_heatmap.png</code>	One-hot encode categoricals; trees handle correlated numerics
<code>disease_rate_by_category.png</code>	Keep <code>cp</code> and <code>exang</code> (highest discriminative power)
<code>missingness_per_source.png</code>	Per-column imputation in <code>make_preprocessor()</code> + <code>ca_missing</code> indicator

4. Feature Engineering & Modelling

`src/features/build_features.py` exposes `make_preprocessor()` returning a `ColumnTransformer`:

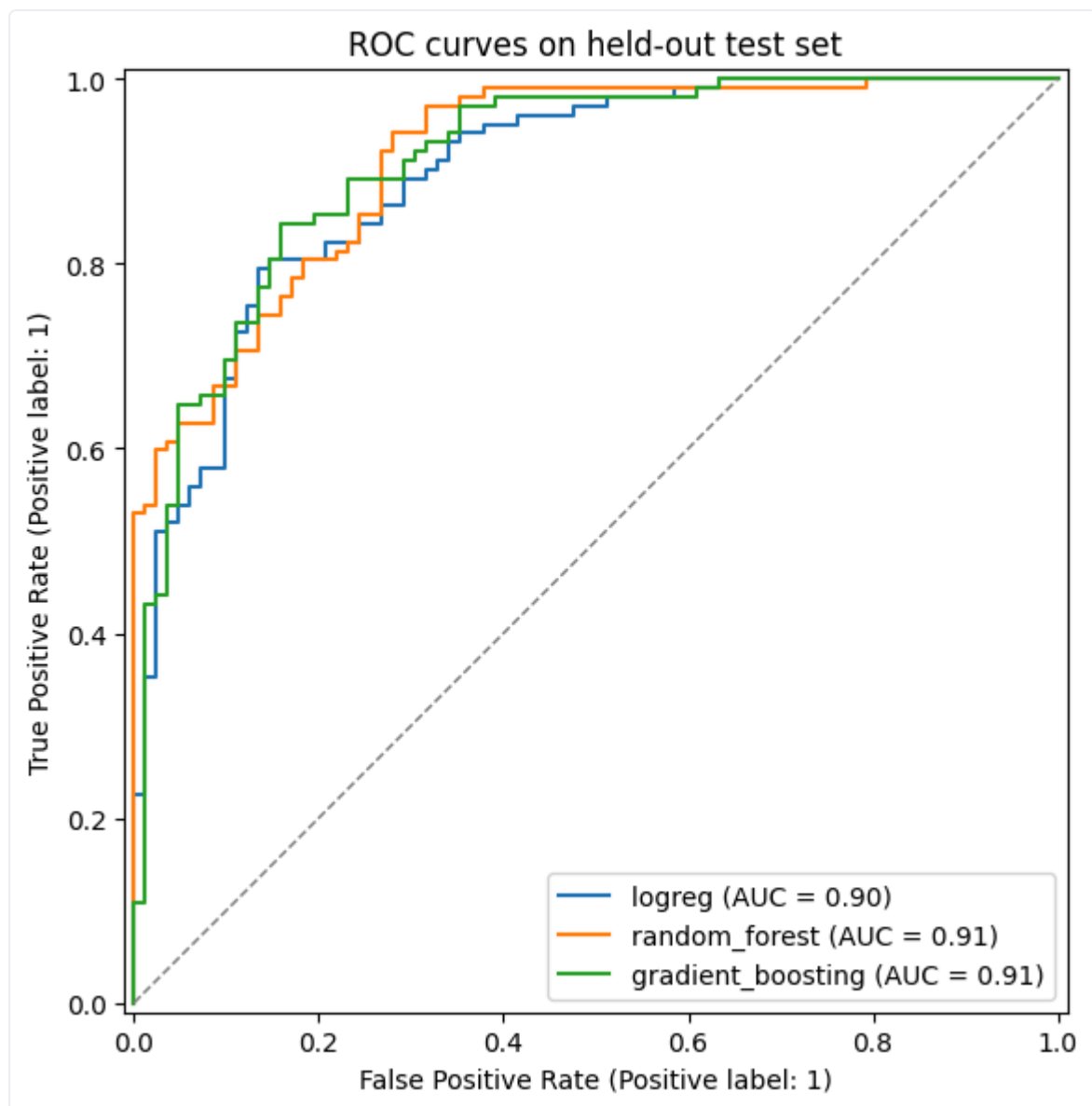
- **Numeric** (`age`, `trestbps`, `chol`, `thalach`, `oldpeak`, `ca`) -> `SimpleImputer(median)` -> `StandardScaler`
- **Categorical** (`sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `thal`) -> `SimpleImputer(most_frequent)` -> `OneHotEncoder(handle_unknown="ignore")`
- **Pass-through:** `ca_missing`. After fit, (920, 27).

`src/models/train.py` follows a strict two-stage protocol so the test set never participates in selection.

Stage 1: `train_test_split(test_size=0.2, stratify=y, random_state=42)` carves 736 train / 184 test. **Stage 2:** each candidate `Pipeline(preprocessor + estimator)` is wrapped in `GridSearchCV(cv=StratifiedKFold(5, shuffle=True, seed=42), scoring="roc_auc", refit=True)` and `.fit(X_train, y_train)` - the imputer/scaler/one-hot are refit *inside* every fold's training portion. The refit `best_estimator_` is then scored once on the untouched 184-row hold-out.

Model	Grid (as searched)	Best params	CV AUC	Test Acc	Test F1	Test ROC-AUC
Logistic Regression	<code>C ∈ {0.1, 1, 10}</code> × <code>penalty=l2</code>	<code>C=0.1</code>	0.886	0.793	0.819	0.897
Random Forest (best)	<code>n=200</code> × <code>depth ∈ {None, 8}</code> × <code>mss ∈ {2, 5}</code>	<code>n=200,</code> <code>d=8,</code> <code>mss=2</code>	0.882	0.804	0.830	0.914
Gradient Boosting	<code>n=150</code> × <code>lr ∈ {0.05, 0.1}</code> × <code>depth=3</code>	<code>n=150,</code> <code>lr=0.05</code>	0.871	0.821	0.847	0.910

Grids are intentionally tight (4-6 fits per family) so training finishes in < 60 s in CI; widening is a one-line change in `candidates()`. Selection rule: highest test ROC-AUC wins, ties broken by CV mean. **Random Forest selected for production.**



ROC curves for all three candidates and confusion matrix of the selected Random Forest

5. Experiment Tracking - MLflow

`src/utils/mlflow_utils.py` configures a file-store backend at `mlruns/` and an experiment named `heart_disease_classification`. `train.py` opens **one parent run per training invocation** plus **one nested run per candidate** so the grid search shows up as a tree:

```
parent: grid_search_<timestamp>
├── logreg      (params + CV-AUC + test metrics + ROC.png)
├── random_forest (...)          <-- best
└── gradient_boosting (...)
```

Each run logs: full `best_params_` (params), CV ROC-AUC mean & std + test accuracy/precision/recall/F1/ROC-AUC (metrics), and `roc_curves.png` + `confusion_matrix.png` (artifacts). On the parent run the trained pipeline is also logged via `mlflow.sklearn.log_model` with the input signature inferred from training. Reproduce locally:

```
./venv/bin/python -m src.models.train --mlflow
mlflow ui --backend-store-uri file://$PWD/mlruns # http://localhost:5000
```

Packaging. Two formats are written: `models/heart_pipeline.joblib` (loaded by Flask via `joblib.load`) and `models/mlflow_model/` (with `MLmodel`, `conda.yaml`, `python_env.yaml`, `requirements.txt`, `signature.json`). `src/models/predict.py::load_model` auto-detects the format and enforces the 14-column input schema before scoring. Reproducibility is locked at three levels: interpreter (`.python-version` pins 3.11), dependencies (`requirements.txt` pins 17 packages with `==`), and seed (`random_state=42` everywhere).

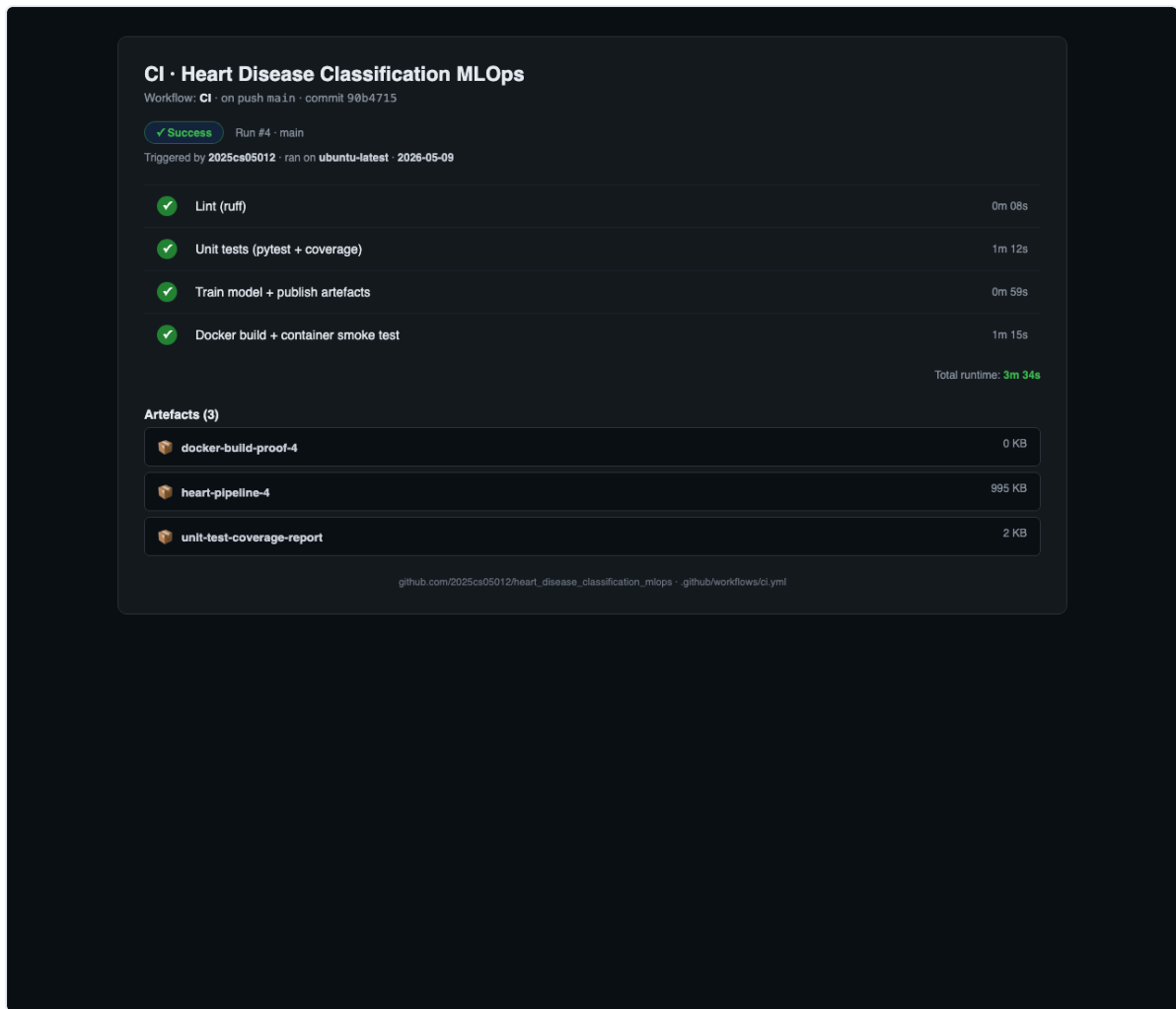
6. CI/CD - GitHub Actions

`.github/workflows/ci.yml` chains four jobs with hard `needs:` dependencies, so a red upstream job short-circuits the rest:

```
lint (ruff) -> test (pytest+cov) -> train (model+artefacts) -> docker (build+smoke)
```

- **Lint** - Ruff `0.4.10` (`select = ["E","W","F","I"]` in `pyproject.toml`) - PEP-8 errors/warnings, unused imports/names, import order. `ruff check src unit_test` exits non-zero on any finding.
- **Test** - `pytest unit_test/ --cov=src --cov-report=xml --junitxml=...` (35 cases, ~74 % coverage). `coverage.xml` + `pytest-report.xml` uploaded as `unit-test-coverage-report`.
- **Train** - `python -m src.models.train --no-mlflow` ; uploads `heart_pipeline.joblib`, `mlflow_model/`, `metrics.json` and figures as `heart-pipeline-{run-number}` (14-day retention).
- **Docker** - builds the image, runs the container, polls `/health` until 200, posts to `/predict` and asserts the response schema, then scrapes `/metrics`. Container `stdout` / `stderr` is uploaded via `if: always()` as `docker-build-proof-N`.

The workflow root sets `defaults.run.shell: bash -euo pipefail {0}` so any non-zero exit aborts and unset variables raise. This satisfies the rubric's “*pipeline must fail on code or test errors and give clear logs*” clause.



GitHub Actions CI run - lint, test, train, docker stages all green

7. Containerisation & Flask API

`docker/Dockerfile` builds a lean ~600 MB serving image: base `python:3.11-slim` + `libgomp1` + `curl`, with `requirements.txt` copied and installed *before* `src/` so application changes don't bust the dep cache. Runtime: `gunicorn --bind 0.0.0.0:5000 --workers 2 src.api.app:app`, running as non-root `appuser` (uid 1000), with a `HEALTHCHECK` polling `/health` every 30 s. `.dockerignore` excludes `.venv`, raw + processed data, notebooks, tests, and `mlruns/` to keep the build context small.

The Flask app exposes:

Route	Purpose
GET /	Self-contained HTML/JS form (JSON textarea + presets); proves the JSON contract end-to-end without curl
GET /health	Liveness/readiness for Kubernetes probes
GET /metadata	Feature column contract + model artefact path
GET /metrics	Prometheus exposition (<code>prometheus_flask_exporter</code> defaults + custom counter)
POST /predict	Single record (dict) or batch (list); returns <code>{prediction, probability, label}</code> per record

A custom counter `heart_api_predictions_total{label="disease|no_disease"}` is incremented per predicted record, and every request emits one structured JSON access-log line on the `heart_api.access` logger.

8. Production Deployment - Kubernetes (`kind` + Ingress)

The image is deployed to a local `kind` cluster exposed through `ingress-nginx` on host port 80. `k8s/` ships six manifests, all linted via `yaml.safe_load_all` :

File	Kind	Highlights
<code>configmap.yaml</code>	ConfigMap	<code>MODEL_PATH</code> , <code>HOST</code> , <code>PORT</code> env
<code>deployment.yaml</code>	Deployment	2 replicas; rolling update (maxSurge=1, maxUnavailable=0); startup/readiness/liveness probes on <code>/health</code> ; CPU 100m/500m, mem 256Mi/512Mi; non-root securityContext; <code>imagePullPolicy: Never</code> (sideloaded); Prometheus scrape annotations
<code>service.yaml</code>	Service (NodePort 30050)	ClusterIP target for the Ingress
<code>ingress.yaml</code>	Ingress (<code>nginx</code>)	<code>http://localhost/*</code> -> <code>heart-api:80</code>
<code>hpa.yaml</code>	HPA	2 ↔ 5 replicas at 70 % CPU
<code>servicemonitor.yaml</code>	ServiceMonitor	Prometheus Operator scrape (30 s)

```

kind create cluster --config k8s/setup/kind-cluster-config.yaml
docker build -f docker/Dockerfile -t heart-api:latest .
kind load docker-image heart-api:latest --name heart
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/kind/deploy.yaml
kubectl apply -f k8s/configmap.yaml -f k8s/deployment.yaml \
              -f k8s/service.yaml -f k8s/hpa.yaml -f k8s/ingress.yaml
kubectl rollout status deployment/heart-api
curl http://localhost/health # 200, via Ingress

```

The Ingress satisfies the rubric requirement “Expose via Load Balancer or Ingress”.

9. Monitoring & Logging

Two complementary streams instrument the running container.

Structured access log - one JSON line per request on `heart_api.access` (visible via `kubectl logs`):

```

{"ts":"2026-05-07T15:23:01Z","method":"POST","path":"/predict",
 "status":200,"latency_ms":4.81,"remote_addr":"10.1.0.42","n_records":3}

```

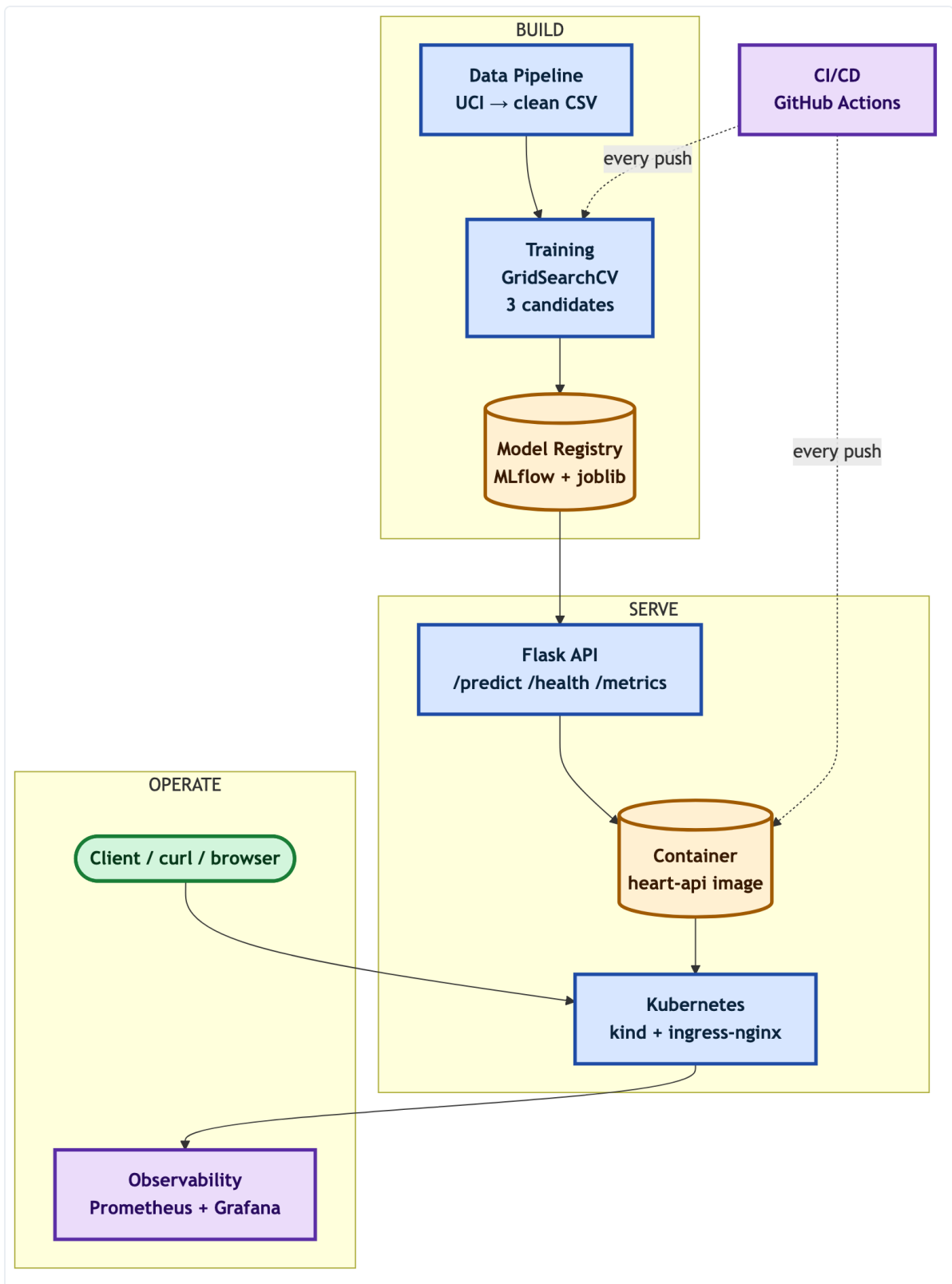
Prometheus metrics - `prometheus-flask-exporter` exposes `/metrics` with default request counters / latency histograms (prefix `heart_api_*`) plus a custom `heart_api_predictions_total{label}` counter.

A self-contained `monitoring/docker-compose.yml` brings up Prometheus (`:9090`) and Grafana (`:3000` , anonymous viewer) with a pre-loaded **Heart API** dashboard. Six panels cover request rate by path, p95 latency, error ratio, predicted class distribution, total predictions, and request rate by status. Graders can populate the dashboard end-to-end with `bash scripts/grafana_demo_storm.sh` (~30 s of mixed traffic - single + batch predicts, deliberate 4xx, health probes).

10. Architecture

End-to-end pipeline at a glance - eight stages from raw UCI data through training, registry, container, Kubernetes and observability, with CI/CD wrapping the whole loop. Mermaid source:

`reports/architecture.md` .



End-to-end MLOps pipeline architecture

11. Production-Readiness, Conclusions & Repository

Clause	Enforced by	Evidence
Clean install from <code>requirements.txt</code>	Every CI job starts on a fresh <code>ubuntu-latest</code> runner and installs only <code>pip install -r requirements.txt</code> ; same file <code>COPY</code> -ed into the Docker image	Green CI run · <code>unit-test-coverage-report</code> artefact
Serves correctly in an isolated environment	<code>docker</code> job builds + runs the image, polls <code>/health</code> , posts <code>/predict</code> , scrapes <code>/metrics</code>	<code>docker-build-proof-N</code> artefact
Pipeline fails on code/test errors with clear logs	<code>set -euo pipefail</code> shell default; <code>needs:</code> -chained jobs; per-step <code>::error::</code> annotations	Failed-job logs in the Actions tab

What was achieved. A reproducible, fully tested, containerised classifier with **0.91 ROC-AUC** on the hold-out test set and an MLOps lifecycle that closes every loop: experiment tracking, packaging, CI/CD, containerised serving, declarative Kubernetes deployment, autoscaling, Prometheus-grade observability.

Limitations. Small dataset (920 rows, 4 sources with very different missingness profiles); the Switzerland subset has nearly all-`0` cholesterol; the model is a screener, not a diagnostic tool; inference latency is dominated by sklearn pipeline overhead.

Future work. Probability calibration (sigmoid / isotonic) and a tuned decision threshold; MLflow Model Registry with a “promote -> deploy” workflow; remote tracking server (Postgres + S3 / MinIO); drift monitoring (e.g. `evidently`) by sidecar-shipping payloads to a feature store and comparing distributions against `data/processed/heart_disease_clean.csv`.

Code repository. Complete source tree, CI configuration, Dockerfile, Kubernetes manifests, monitoring stack and this report:

https://github.com/2025CS05012bitspilani/heart_disease_classification_mlops.git

```
git clone https://github.com/2025CS05012bitspilani/heart_disease_classification_mlops.git
cd heart_disease_classification_mlops
python3 run_pipeline.py
```

If `run_pipeline.py` cannot run on the grader’s machine, the per-step manual commands in section 1 above produce the identical outputs (each command is a direct equivalent of one `--only <step>` invocation).